

navAjna Walk-In — the one-page version

navAjna walk-in · HITEC City, Hyderabad · Learn nothing new today — revision only.

Read the night before, and again in the auto on the way.

Carry these — the JD lists them explicitly

- [] **Updated resume** — 3 printed copies
- [] **Government photo ID** — Aadhaar or PAN
- [] **Education certificates** — B.Tech degree and marks memos
- [] Payslips / offer letter / relieving letter from Zensar, in case they ask
- [] Pen and a small notebook
- [] Phone charged, with your GitHub and portfolio reachable — **it is face to face, so you can actually show your work**
- [] Water

Address: Level 3 & 4, Karan Arcade, Plot #78, Patrika Nagar, HITEC City, Hyderabad 500081.

Leave early. HITEC City traffic is unpredictable and arriving flustered costs you the first ten minutes. Both rounds are the same day, so expect to be there a while — eat first.

The 10 answers that carry this interview

1. **HashMap** — `hashCode()` → spread `h ^ (h >> 16)` → bucket `(n-1) & hash`; collisions chain, over 8 entries becomes a red-black tree; capacity 16, load factor 0.75, resizes at 12.
2. **equals/hashCode** — equal objects must have equal hash codes, or the map looks in the wrong bucket and never finds the entry.
3. **Re-renders** — own state · props · **parent re-rendered** · context. Fix with `React.memo` → `useCallback` / `useMemo` → move state down.
4. **key** — stable identity. Index breaks on delete or reorder; React reuses the wrong DOM node and the typed value stays on the wrong row.
5. **Four states** — loading, error, **empty**, success. Empty is the one people forget.
6. **fetch does not throw on 404 or 500** — check `res.ok` and throw yourself.
7. **JWT** — header.payload.signature, **signed not encrypted**, nothing sensitive inside. Authorisation must be server-side; a hidden button is not security.
8. **N+1** — one query for the list, one per row for the relation. `JOIN FETCH`, `@EntityGraph`, batch fetch, or a DTO projection.
9. **@Transactional / AOP proxy** — an internal `this.method()` call bypasses the proxy, so nothing fires.
10. **Microservices** — independent deploy, scale, fault isolation; **cost** is network failure, no distributed transaction, harder debugging. *Say the cost.*

Memory — the backbone answer •

Stack = one per thread, holds method frames and local variables, cleared automatically when the method returns.

Heap = one, shared, holds every object, cleaned by the GC.

Primitives hold the value. Objects hold a reference — an address into the heap.

```
void change(int n, StringBuilder s) { n = 99; s.append("world"); }  
// n unchanged (copied the value) · s changed (copied the address)  
// -> Java is always pass-by-value; for objects the value IS the address
```

- **StackOverflowError** = recursion with no base case · **OutOfMemoryError** = too many live objects, usually something unbounded.
- **A Java memory leak** = keeping a reference you no longer need, so the GC is not allowed to collect it.
- **ArrayList** = one array, compact, cache friendly · **LinkedList** = a separate node per item with two pointers, ~3× the overhead, scattered.
- **HashMap holds** a table array + **one Node per entry** + deliberately empty slots (0.75 load factor). Resizing allocates a new array **twice the size** while the old one still exists.
- **String pool** — literals are shared, `new String()` is a separate object. `a == b` true, `a == c` false.
- **+ in a loop** — new String object every iteration, $O(n^2)$ and garbage. Use `StringBuilder`.
- **Integer cache** -128..127. `int` is a stack value; `Integer` is a heap object.
- **Spring singleton** — one object in the heap for all requests; locals are per-request stack, **instance fields are shared**.
- **JPA persistence context** keeps the entity **plus a snapshot** for dirty checking — two copies per row, which is why batch jobs OOM. DTO projections avoid it.
- **React:** mutating state keeps the same reference so no re-render; a new function each render is a new heap object, which is why `useCallback` exists; **an uncleaned timer or listener holds the whole component alive** — that is what a React leak is.

Scenario questions — the five-step method •

Clarify → Diagnose → Causes → Fix + cost → Prevent.

Most candidates jump to the fix. The marks are in **diagnose** (say how you would find out before saying what you would change) and **prevent** (a test, a log, an alert, so it cannot recur). Almost nobody does the last one.

The line to reuse: *"Before I change anything I would want to know where the time is actually going — my first guess is usually wrong."*

The five most likely, all from your own work:

- **Slow in prod, fast in testing** → $N+1$. Count the queries in the SQL log. Join fetch, `@EntityGraph`, or a DTO projection. *Index cost: slows writes.*
- **Two approvers at once** → lost update. `@Version` optimistic locking; the second write hits zero rows and you tell them it is already actioned.
- **Missing audit record** → the **AOP proxy**: an internal `this.method()` call never goes through the proxy, so the aspect never fires. Same caveat as `@Transactional`.
- **Slow list page** → server-side pagination first; virtualise if it must be one list; memoise the row.
- **Legacy codebase, no tests** → read one request end to end, make one small safe change, then add tests **as you touch things**, not as a project.

Your numbers — say them without hesitating

20,000+ corporate clients · **Lighthouse 62 → 88** · **60%** off card processing ·
50,000+ audit transactions a day · **1,000+** monthly reports ·
14-screen onboarding flow · **7-screen** recall workflow · **3+** features reusing your components ·
40% fewer re-renders on QKart · Core Web Vitals **65 → 82** at Pixelvide.

The three sentences that decide this

Opening: *"...and I am available to join immediately, so the seven-day window is not a problem for me."* **Say this in the first two minutes.**

The gap: two sentences, the real reason, then **specifically** what you have done since

February — Crio.Do, anything you have built, the GenAI certifications. Then pivot back to availability.

Track: *"Frontend primarily — React, Redux and TypeScript are my day-to-day. But I built the Spring Boot backends for the same applications, so I can work across both."*

In the room

- **Structure your answers.** "Three things —" then give three.
- **Say your answer and stop.** Over-explaining is the commonest nervous habit.
- **Think out loud in the coding question.** Silence reads as stuck.
- **Ask the edge cases before you code.** Null, empty, duplicates.
- **State the complexity before they ask.**
- **If you do not know it:** *"I have not used that directly — my understanding is X. Is that the direction you mean?"* Never bluff; the second question always comes.
- **Ask them questions.** Round 2 is fitment; make it a conversation.

The last five minutes before you walk in

You have shipped five production features on a banking platform used by 20,000 clients, both ends of the stack, with real numbers attached. For a role asking 1–3 years, that is a strong profile — and you can start next week, which most of the queue cannot.

Sit up. Breathe out slowly. Smile before you speak.

Go get it.